

Aula 3 — Interfaceamento Paralelo e Display

O controlador HD44780 e o custo da espera

Microcontroladores — DENE/UFMT

Objetivos da aula

- Distinguir comunicação paralela e serial e reconhecer o compromisso entre número de pinos e velocidade.
- Descrever a organização do controlador HD44780: memória de exibição, memória de caracteres e a separação entre comando e dado.
- Implementar a comunicação em quatro bits, incluindo a sequência de inicialização e as temporizações exigidas.
- Estimar o tempo consumido por uma atualização de tela e reconhecê-lo como o principal consumidor de tempo do sistema.
- Aplicar estratégias de atualização que evitam tremulação e reduzem o tempo gasto com o display.

1 Paralelo e serial

Ligar um periférico externo ao microcontrolador exige transportar bits. Há duas formas, e a escolha entre elas é um compromisso permanente em projeto embarcado.

Definição --- comunicação paralela

Transmissão simultânea de vários bits, cada um em uma linha física própria, acompanhada de linhas de controle que indicam quando o dado é válido.

A alternativa serial envia os bits em sequência por uma única linha. O paralelo é mais rápido para a mesma frequência de sinal e mais simples de gerar; em compensação, consome pinos — e pinos são, neste dispositivo, o recurso mais escasso de todos. Um display em oito bits ocupa onze pinos com sinais de controle; o mesmo display em quatro bits ocupa sete; um display serial ocupa dois ou três.

Observação

A conta de pinos decide o projeto do semestre. Com 35 pinos e a necessidade de acomodar sensor, aquecedor, ventoinha, teclado matricial e comunicação, os quatro pinos economizados pelo modo de quatro bits não são detalhe: são o que torna o projeto viável no kit. É por isso que o modo reduzido é o padrão neste tipo de aplicação, apesar de exigir mais código.

2 O controlador HD44780

O display alfanumérico do laboratório não é uma matriz de pontos comandada diretamente: é um módulo com controlador próprio, que recebe comandos e caracteres e cuida da varredura, da geração de caracteres e da atualização.

Elemento	Função
Memória de exibição	Guarda os códigos dos caracteres mostrados. Cada posição da tela corresponde a um endereço fixo

Elemento	Função
Gerador de caracteres em ROM	Contém os desenhos dos caracteres padrão, indexados pelo código recebido
Gerador de caracteres em RAM	Permite definir caracteres próprios — tipicamente o símbolo de grau, ausente do conjunto padrão
Registrador de instrução	Recebe comandos: limpar, posicionar cursor, configurar modo
Registrador de dados	Recebe os códigos dos caracteres a exibir

Tabela 1: Elementos internos do controlador.

Três linhas de controle governam a comunicação:

Linha	Nome usual	Significado
RS	Seleção de registrador	Nível baixo indica comando; nível alto indica dado a ser exibido
R/W	Leitura ou escrita	Nível baixo escreve. Em geral ligado permanentemente ao terra, economizando um pino
E	Habilitação	O dado é capturado na borda de descida deste sinal. É o pulso que efetiva a transferência

Tabela 2: Linhas de controle.

Atenção

Ligar a linha de leitura ao terra economiza um pino e tem uma consequência: torna-se impossível ler o indicador de ocupado do controlador. Sem ele, não há como perguntar se a operação anterior terminou — resta esperar o tempo máximo especificado. Toda a temporização por atraso fixo descrita adiante é consequência dessa escolha de hardware.

2.1 Endereçamento da tela

Os endereços da memória de exibição não são contíguos entre as linhas. Numa tela de duas linhas por dezesseis colunas, a primeira linha começa no endereço zero e a segunda começa em 64, independentemente do número de colunas.

Atenção

A descontinuidade é a origem de um defeito clássico: escrever dezessete caracteres seguidos não faz o texto passar para a segunda linha — ele continua em posições invisíveis da primeira. Mudar de linha exige um comando explícito de posicionamento.

3 O modo de quatro bits

No modo reduzido, cada byte é enviado em duas etapas: primeiro os quatro bits mais significativos, depois os quatro menos significativos, cada metade acompanhada de seu próprio pulso de habilitação.

Código

```
static void lcd_pulso(void)
{
    LATCbits.LATC0 = 1;          /* E em nivel alto */
    __delay_us(1);
    LATCbits.LATC0 = 0;          /* borda de descida: captura */
    __delay_us(50);              /* tempo de execucao do comando */
}

static void lcd_nibble(uint8_t valor)
{
    LATD = (uint8_t)((LATD & 0x0F) | (valor & 0xF0));
    lcd_pulso();
}

void lcd_escrever(uint8_t valor, uint8_t eh_dado)
{
    LATCbits.LATC1 = eh_dado;     /* RS */
    lcd_nibble(valor);           /* parte alta */
    lcd_nibble((uint8_t)(valor << 4)); /* parte baixa */
}
```

Observação

Note o uso de LATD na leitura e na escrita da primeira função: preserva-se a parte baixa do latch enquanto se altera a parte alta. Ler PORTD ali reintroduziria exatamente o defeito de leitura-modificação-escrita do encontro

2. A regra da aula anterior não é abstrata — ela aparece na terceira linha de código do primeiro periférico externo do curso.

3.1 A sequência de inicialização

O controlador inicia em modo de oito bits. Configurá-lo para quatro exige uma sequência específica, executada às cegas: enviam-se meias palavras de comando antes de o controlador estar no modo em que essas palavras fazem sentido.

Atenção

Essa sequência tem aparência de superstição — valores repetidos, esperas arbitrárias — e é frequentemente copiada sem compreensão. A lógica é a seguinte: o comando de configuração é reconhecido tanto em oito quanto em quatro bits; repeti-lo em modo de oito bits garante um estado conhecido, qualquer que fosse o estado inicial, e só então se comuta para quatro bits. Os atrasos iniciais existem porque o controlador tem sua própria inicialização interna, mais lenta que a do microcontrolador.

Código

```
void lcd_iniciar(void)
{
    __delay_ms(50);              /* espera a inicializacao interna do modulo */
}
```

```
    lcd_nibble(0x30); __delay_ms(5);
    lcd_nibble(0x30); __delay_us(150);
    lcd_nibble(0x30); __delay_us(150);
    lcd_nibble(0x20);          /* a partir daqui, modo de 4 bits */

    lcd_comando(0x28);          /* 4 bits, 2 linhas, matriz 5x8 */
    lcd_comando(0x0C);          /* display ligado, sem cursor */
    lcd_comando(0x06);          /* avanco automatico do cursor */
    lcd_comando(0x01);          /* limpar */
    __delay_ms(2);              /* limpar demora mais que os demais */
}
```

4 O custo em tempo

Derivação

A maioria dos comandos exige cerca de 37 μ s. Escrever dezesseis caracteres numa linha custa

$$16 \cdot 37 \mu\text{s} \approx 0,6 \text{ ms}$$

Acrescentando o posicionamento de cursor de cada linha e considerando duas linhas, uma atualização completa de tela custa da ordem de 1,3 ms. O comando de limpeza, sozinho, exige aproximadamente 1,5 ms — quarenta vezes mais que um comando comum.

Atenção

Compare com os números do curso: uma conversão do conversor analógico-digital custa 15 μ s, e o tratador de interrupção do encontro 9 custa poucos microssegundos. **O display é, com folga, o periférico mais lento do sistema** — duas ordens de grandeza acima de tudo o mais. Qualquer discussão sobre desempenho neste projeto passa por ele.

4.1 Duas consequências de projeto

Não limpar a tela a cada atualização. Limpar e reescrever custa os 1,5 ms do comando somados ao tempo de escrita, e produz tremulação visível: por um instante a tela está em branco. A prática correta é reposicionar o cursor e sobrescrever, mantendo o comprimento do texto constante e completando com espaços quando o número encolhe — caso contrário, restam dígitos antigos à direita.

Não atualizar mais rápido do que se consegue ler. Atualizar a tela a cada ciclo de 1 ms é desperdício puro: o olho não acompanha, e o custo é o mais alto do sistema. Uma atualização a cada 200 ms a 500 ms é confortável e reduz o consumo de tempo em duas ordens de grandeza.

Código

```
/* Escrever apenas quando o valor mudou. */
static int16_t exibido = INT16_MIN;

void atualizar_display(int16_t decimos)
{
    if (decimos == exibido) {
        return;          /* nada a fazer: sai imediatamente */
    }
}
```

```
    }  
    exibido = decimos;  
  
    lcd_posicao(0, 0);  
    lcd_texto("T=");  
    lcd_numero_fixo(decimos);      /* largura constante */  
    lcd_texto(" C ");  
}
```

Observação

Essa técnica — guardar o que já foi mostrado e escrever apenas a diferença — é a versão elementar de uma ideia que reaparece em toda interface gráfica, de displays de duas linhas a navegadores. O princípio é o mesmo: **comparar com o estado exibido é sempre mais barato que redesenhar**.

5 Transposição

Displays modernos são majoritariamente seriais, e a economia de pinos é decisiva. Mudam a camada física e a velocidade; permanece intacta a separação entre comando e dado, e permanece o fato de o display ser o periférico lento do sistema.

Observação

Em plataformas com mais memória, aparece uma diferença estrutural: o **quadro de memória**. Em vez de escrever caracteres diretamente no display, o programa desenha numa imagem em RAM e a transfere de uma vez, por acesso direto à memória, sem custo de processador. Isso exige memória que este dispositivo não tem — um display gráfico modesto consome mais RAM do que os 2 KB disponíveis — e é uma das fronteiras concretas entre 8 e 32 bits.

6 Exercícios

Exercício 3.1

Calcule quantos pinos são necessários para o display em oito bits e em quatro bits, incluindo as linhas de controle e considerando a linha de leitura ligada ao terra. Em seguida, verifique no contrato de pinos do laboratório se a versão de oito bits seria viável junto aos demais periféricos do projeto.

Exercício 3.2

Um estudante escreve `lcd_texto("Temp: 25.3 C")` e, em seguida, `lcd_texto("Temp: 9.1 C")`, sem limpar a tela. Descreva exatamente o que aparece no display e proponha duas correções, uma na função de escrita e outra na formatação do número.

Exercício 3.3

Estime o tempo total gasto com o display em um sistema que atualiza duas linhas completas a cada 100 ms, e expresse esse valor como percentual do tempo de processador. Repita para atualização a cada 500 ms e comente a diferença.

Exercício 3.4

Explique por que a função `lcd_nibble` apresentada lê LATD em vez de PORTD, e descreva concretamente o defeito que ocorreria com a leitura da porta caso os quatro pinos baixos estivessem ligados a uma carga capacitiva.

Exercício 3.5

A sequência de inicialização envia três vezes o mesmo valor antes de comutar o modo. Justifique tecnicamente essa repetição, considerando que o microcontrolador pode ser reiniciado sem que o display seja desenergizado — e que, nesse caso, o controlador pode estar em qualquer um dos dois modos.